CO-3 Assessment Tool - 1

Name : Tamil Elakiya S

Reg No : 192524443

Experiment: Race Condition in Online Auction Bidding

Aim

To identify and eliminate a check-then-act race condition in an online auction system using per-lot synchronization and AtomicReference with CAS.

Problem Statement

When two threads place bids for the same lot at nearly the same time, the lower bid may overwrite the higher bid because the following operations are not atomic:

```
if (bid.getAmount() > lot.getCurrentHighBid()) {
   lot.setCurrentHighBid(bid.getAmount());
   lot.setHighBidder(bid.getBidderId());
}
```

For example:

Current bid = ₹1000

Thread 1 → ₹1500

Thread 2 → ₹1200

Per-Lot Lock

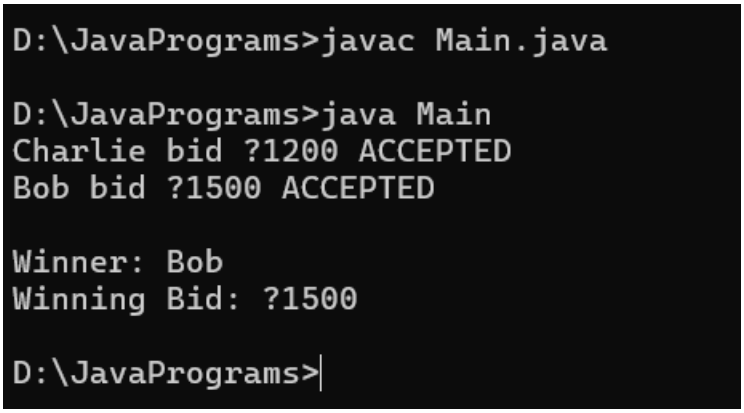
```
class Auction {

   public static void placeBid(Lot lot, Bid bid) {

      synchronized (lot) {

         if (bid.amount > lot.currentHighBid) {
            lot.currentHighBid = bid.amount;
            lot.highBidder = bid.bidder;
            System.out.println(bid.bidder +
                  " bid ₹" + bid.amount + " ACCEPTED");
         } else {
            System.out.println(bid.bidder +
```

```
                    " bid ₹" + bid.amount + " REJECTED");
            }
        }
    }
}

class Lot {
    int currentHighBid = 1000;
    String highBidder = "Alice";
}

class Bid {
    String bidder;
    int amount;

    Bid(String bidder, int amount) {
        this.bidder = bidder;
        this.amount = amount;
    }
}
```

Test

```
public class Main {
    public static void main(String[] args) throws Exception {

        Lot lot = new Lot();

        Thread t1 = new Thread(() ->
            Auction.placeBid(lot, new Bid("Bob", 1500)));

        Thread t2 = new Thread(() ->
            Auction.placeBid(lot, new Bid("Charlie", 1200)));

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("\nWinner: " + lot.highBidder);
        System.out.println("Winning Bid: ₹" + lot.currentHighBid);
    }
}
```

Output



Lock-Free AtomicReference

```
import java.util.concurrent.atomic.AtomicReference;

record BidState(int amount, String bidder) {}

class LockFreeAuction {

   AtomicReference<BidState> highest =
      new AtomicReference<>(new BidState(1000, "Alice"));

   boolean placeBid(int amount, String bidder) {

      while (true) {

         BidState current = highest.get();

         if (amount <= current.amount())
            return false;

         BidState updated =
            new BidState(amount, bidder);

         if (highest.compareAndSet(current, updated))
            return true;
      }
   }
}
```

Output

```
D:\JavaPrograms>javac Main2.java

D:\JavaPrograms>java Main2
Bob bid ?1500 ACCEPTED
Charlie bid ?1200 REJECTED

Winner: Bob
Winning Bid: ?1500

D:\JavaPrograms>
```

Conclusion

The race condition is eliminated by making the **check + update atomic**.

- **Per-lot lock:** Simple, reliable and suitable for high contention.
- **AtomicReference + CAS:** Lock-free and potentially faster, but repeated retries can occur.